الطلب والجواب في عالم الأبواب

Lectures 1–2 — Introduction to Web Application Security & HTTP Protocol Deep Dive

> **Web application** : software that runs on a remote server and is accessed through a browser (Gmail, online banking, a university portal). None of its code runs permanently on your machine.

Q/ What is the request/response cycle ?

- The browser sends a request → the server processes it (often talking to a database) → it sends back a response that the browser turns into the page. This happens on every click, form submit or page load.

Q/ What four steps happen before the browser "talks" to the application ?

1. **DNS resolution**: the name (example.com) becomes an IP address.
2. **TCP handshake**: SYN / SYN-ACK / ACK opens a reliable connection.
3. **TLS handshake** (HTTPS): both sides agree on encryption keys — confidentiality, integrity, and server authentication through certificates.
4. **HTTP request/response**: only now does the application conversation begin (GET /login …).

Network security	Web application security
Protects the pipes: firewalls, routers, VPNs, TLS	Protects what runs **on top**: the application logic itself
Focus: how data travels	Focus: what the application **does** with data once it arrives

A firewall won't stop an attacker who logs in normally and then manipulates a form field to see another user's data — the traffic looks legitimate.

Q/ Split the attack surface of a web application.

Client-side attacks (target the victim's browser)	Server-side attacks (target app server, database, backend)
Cross-Site Scripting (XSS) — Week 6	SQL Injection — Week 9

Client-side attacks (target the victim's browser)	Server-side attacks (target app server, database, backend)
Cross-Site Request Forgery (CSRF) — Week 7	Command Injection
DOM-based manipulation	Server-Side Request Forgery (SSRF)
Clickjacking	Broken Access Control — Week 12

Q/ Why are web apps attractive targets ?

1. **Publicly reachable**: anyone with a browser can reach the login page.
2. **Valuable data**: passwords, payment details, personal records.
3. **Complex**: more code = more places for a small mistake to hide.

> **Confidentiality** : only authorized users can see the data (your bank balance stays private).

> **Integrity** : data isn't changed without permission (no one edits your order total or grade).

> **Availability** : the service stays reachable (a flood of fake requests can't take it offline).

XSS mainly threatens **confidentiality** (session theft); SQL injection can break **confidentiality and integrity**; DoS targets **availability**.

> **The fundamental flaw — mixing code and data** : letting user input be interpreted as **executable instructions** instead of plain data. It is the root of SQL injection, command injection, XSS and more.

“An interpreter that treats user input as code, instead of as data, is already broken.”

Q/ List the OWASP Top 10 (2021) — the edition used in this course.

1. A01 Broken Access Control
2. A02 Cryptographic Failures
3. A03 Injection
4. A04 Insecure Design
5. A05 Security Misconfiguration
6. A06 Vulnerable and Outdated Components
7. A07 Identification and Authentication Failures

8. A08 Software and Data Integrity Failures
9. A09 Security Logging and Monitoring Failures
10. A10 Server-Side Request Forgery (SSRF)

> The 2025 edition reshuffles it (Security Misconfiguration moves to #2; Software Supply Chain Failures is new). Always check the **year** before comparing numbers.

> **Rules of Engagement (RoE)** : a written agreement before testing that defines what is in scope / out of scope, timing, allowed methods and contacts.

> **Responsible (Coordinated) Disclosure** : report a flaw privately to the owner first and give them a fair window (commonly **~90 days**) before going public.

> **Safe Harbor / Bug Bounty** : platforms (HackerOne, Bugcrowd) publish an official, legally protected scope so testing within it is authorized (and often paid).

> "I was just curious" is not a legal defense — **permission first, always**.

HTTP

Q/ What are the four parts of every HTTP message ?

1. **Start line / status line** — what is asked for (request) or what happened (response).
2. **Headers** — metadata: format, size, cookies, caching…
3. **Blank line** — marks the end of headers.
4. **Body** — the payload (form data up, HTML/JSON down).

> **Safe method** : doesn't change server state.

> **Idempotent method** : sending it twice has the same effect as once.

Method	Purpose	Safe?	Idempotent?
GET	Retrieve a resource	Yes	Yes
POST	Create / submit data	No	No
PUT	Replace a resource entirely	No	Yes
PATCH	Partially update	No	No

Method	Purpose	Safe?	Idempotent?
DELETE	Remove a resource	No	Yes
HEAD	Like GET, headers only	Yes	Yes
OPTIONS	Ask what's allowed (CORS preflight)	Yes	Yes

GET parameters live in the **URL** → browser history, server logs, proxy logs. Never put passwords or tokens in a GET query string; use POST.

> **Verb tampering (method override abuse)** : bypassing an access rule that checks only the literal method (e.g. blocks POST to /admin/delete but forgets GET/HEAD), or abusing a header like **X-HTTP-Method-Override**. Fix: **authorize the action, not the verb string**.

Q/ What are the HTTP status code classes ?

1. **1xx** Informational — 100 Continue.
2. **2xx** Success — 200 OK, 201 Created.
3. **3xx** Redirection — 301 Moved Permanently, 302 Found.
4. **4xx** Client error — 400, 401, 403, 404.
5. **5xx** Server error — 500, 503.

401 Unauthorized	403 Forbidden
Actually means unauthenticated	Identity known, **permissions insufficient**
"Who are you?"	"I know who you are, and the answer is still no."

> **Username enumeration** : different messages for "wrong password" and "no such user" let an attacker discover valid usernames. Fix: one **generic error** either way.

> **HTTP is stateless** : every request is brand new; the server keeps no memory. Applications rebuild state with **cookies**.

Q/ Which cookie attributes matter for security ?

1. **Secure**: sent only over HTTPS.
2. **HttpOnly**: JavaScript can't read it — limits what XSS can steal.
3. **SameSite**: restricts sending the cookie with cross-site requests — core CSRF defense.
4. **Domain**: which hosts receive it. **Path**: which URL paths.

SameSite value	Behaviour
Strict	Never sent on any cross-site request, even a link click. Strongest; can break cross-site flows
Lax	Sent only with safe top-level navigation (clicking a link), not cross-site POSTs, images, iframes. **Modern browser default**
None	Sent with every cross-site request; **requires Secure**

Security header	Protects against
Strict-Transport-Security (HSTS)	Forces HTTPS; blocks downgrade / SSL-stripping
Content-Security-Policy (CSP)	Restricts which scripts/resources run — major XSS defense
X-Content-Type-Options: nosniff	Browser guessing file types (content-type confusion)
X-Frame-Options	Being framed by another site — **clickjacking**
Access-Control-Allow-Origin	CORS: which origins may read the response via JS
Referrer-Policy: no-referrer	Leaking the full previous URL (with tokens) to third parties

Q/ How do headers leak data without any "attack" ?

1. **Referer**: a URL like /reset?token=abc123 with a third-party image or link hands the token to that third party → fix with **Referrer-Policy**.
2. **Caching**: without **Cache-Control: no-store**, a bank statement stays in the browser cache after logout; the next user on a shared PC can press Back.

> **Host Header Injection** : the app trusts the client-supplied **Host** header to build absolute URLs.

> **Password-reset poisoning** : the attacker requests a reset for the victim with `Host: attacker.com`; the email link points to the attacker's domain, so the victim's reset **token is sent to the attacker**. Fix: never build security-sensitive URLs from client input; **allow-list** real domains.

> **HTTPS = HTTP + TLS** : the same HTTP conversation wrapped in TLS, giving **confidentiality** (encrypted), **integrity** (can't be altered) and **authenticity** (certificate). Never dismiss a "connection not secure" warning.

Summary of Web Applications Security L1–2: Introduction & HTTP Deep Dive. Built only from the lecture file; check the original slides for anything you're unsure about.